# ONLINE GAME LEVEL EDITOR

M . SHIVA NANDHESWARA REDDY

J . KALYAN RAM

# PROBLEM STATEMENT:-

- The objective is to design a maze-based game level editor using a graph data structure.
  The system allows users to dynamically create, manage, and modify maze levels using CRUD operations through a menu-driven C program.

# REAL-WORLD APPLICATION:-

- Used in game development for designing levels
- Helps in AI pathfinding and navigation
- Applied in robotics for movement planning
- Useful in network routing and mapping systems

# DATA STRUCTURE USED:-

- **Graph (Adjacency List Representation)** Nodes represent rooms
- Edges represent paths
- **Linked List** Stores adjacency list
- **Dynamic Memory Allocation** malloc() and free()

# SYSTEM DESIGN:-

- Maze modelled as a graph structure
- Each room acts as a vertex
- Paths between rooms are edges
- Adjacency list stores connections
- Menu-driven interface for user interaction

# ALGORITHM:-

- Initialise graph with rooms. Add edges to connect rooms. Perform CRUD operations: Create: Add room/path
- Read: Display maze
- Update: Modify connections
- Delete: Remove room/path
- Traverse graph for searching

# IMPLEMENTATION:-

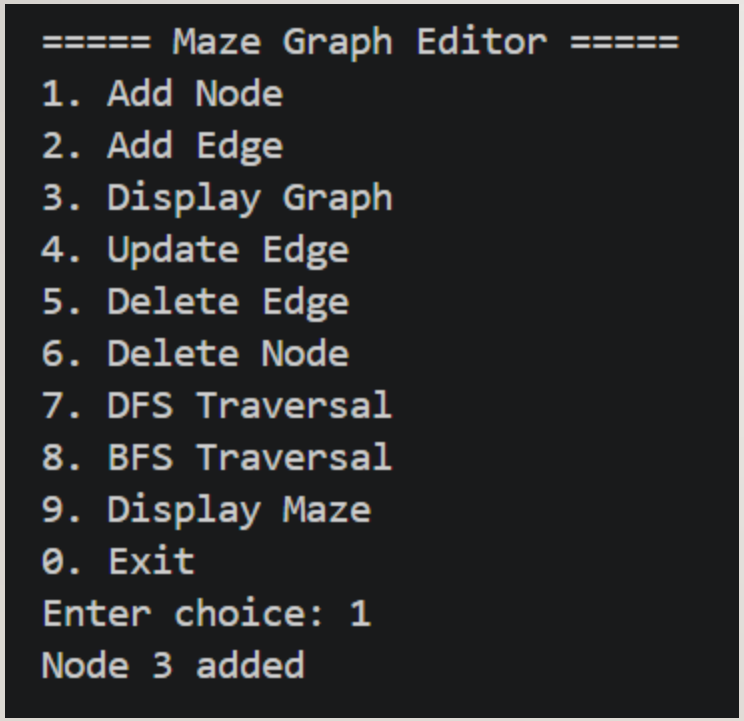
- Programming Language: C Concepts Used: Structures (struct)
- Functions (modular approach)
- Dynamic memory allocation
- Graph using a linked list

# DEMO OUTPUT:-

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 1
Node 0 added

===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 1
Node 1 added
```

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 1
Node 2 added
```




Add Node means **adding a new room (vertex)** to the maze. Each node represents a **point/room in the maze.** Nodes are numbered automatically starting from **0**

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 2
Enter two nodes: 0 1
Edge added between 0 and 1
```

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 2
Enter two nodes: 1 2
Edge added between 1 and 2
```

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 2
Enter two nodes: 2 3
Edge added between 2 and 3
```

- **1 and 2** → Nodes (rooms in the maze)
- **1 (value)** → Means **create or add connection**
- So, the user is saying:
  "Connect node 1 and node 2"

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 3

Adjacency Matrix:
0 1 0 0
1 0 1 0
0 1 0 1
0 0 1 0
```

- Because the graph is **undirected**, connections work both ways:
- Node 1 → Node 2
- Node 2 → Node 1

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 4
Enter nodes and value (0/1): 1 2 1
Edge updated between 1 and 2
```

- Update Edge means **modifying the connection (path)** between two nodes
- It can:
  - **Add a connection** (if value = 1)
  - **Remove a connection** (if value = 0)
- Used to **edit the maze dynamically**

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 5
Enter nodes: 1 2
Edge removed between 1 and 2
```

- Row = source node
- Column = destination node
- Value 1 = connection exists
- graph[1][2] = 1 → connection added
- graph[2][1] = 1 → connection added

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 6
Enter node: 2
Node 2 removed (edges cleared)
```

- Node 1 is connected to Node 2
- Node 2 is connected to Node 1
- So, the maze path is updated correctly

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 7
Enter start node: 0
DFS: 0 1
```

- Start at node **0**
- Go to its connected node → **1**
- From node 1 → go to **2**
- From node 2 → go to **3**
- No more unvisited nodes → stop

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 8
Enter start node: 0
BFS: 0 1
```

- Start at node **0**
- Visit all neighbours of 0 → **1**
- Then visit the neighbours of 1 → **2**
- Then visit neighbours of 2 → **3**

```
===== Maze Graph Editor =====
1. Add Node
2. Add Edge
3. Display Graph
4. Update Edge
5. Delete Edge
6. Delete Node
7. DFS Traversal
8. BFS Traversal
9. Display Maze
0. Exit
Enter choice: 9

Maze View (connections):
Node 0 -> 1
Node 1 -> 0
Node 2 ->
Node 3 ->
```

- **Node 0 → 1:** Node 0 is connected to Node 1
- **Node 1 → 0 2** Node 1 is connected to Node 0 and Node 2
- **Node 2 → 1 3** Node 2 is connected to Node 1 and Node 3
- **Node 3 → 2:** Node 3 is connected to Node 2

# CONCLUSION:-

- Successfully implemented a maze editor using a graph
- Efficient memory usage with an adjacency list
- Supports dynamic CRUD operations
- Can be extended for advanced features

# THANK YOU